Video Transcripts

Module 12: Types of Databases and Database Containerization

Video 1: Introduction (04:34)

As it turns out, the days of one type of database are long gone. Now, there's a multitude of them, and in this section, we're going to be looking at a handful. We'll start off with the traditional. The relational database management system we will then look a document, a great and flexible model to use and expand and stretch on. Then we'll look at key value pair stores, really even easier to use. A great, a great design change when it comes to the type of databases that you can have. Then we'll look at some of these scalable models, the ones that are distributed and can grow a tremendous volume. The ones that learn from a lot of the projects that were done 20 years ago when it comes to scale and being able to truly in a decentralized way, in a horizontal way, meaning by simply throwing hardware at them, can address this tremendous growth.

We'll end up by looking as well at some of the new online offerings where you're really not concerned about how that implementation is done. You're transferring that responsibility to the cloud provider and you're simply using that resource, a very fast and innovative way to do things. Now, because all of this will be done in a multitude of platforms. Some of them are written in C, some of them are written in Java, some of them are written in other languages. It would be tremendously difficult to do this all by hand.

So, because of it, we will be utilizing containers quite a bit and this will give you some great knowledge to walk away with because this is exactly the same way in which this is done when it comes to industry. So, now let's go ahead and jump into databases. As we are well aware, the world is generating a tremendous amount of data. In fact, you see slides such as the following one, emphasizing the volume of data that's going to be produced every day by autonomous vehicles. And interestingly, this projection falls short. I've seen results of a single engine that can generate gigabytes on its own per minute. So, you can see there that there's a tremendous amount that is going to be generated that is being generated at the moment and that we're really still instrumenting. If you take a look at this slide here at this picture of a city, you can see four major sources of data. You can think of the transit systems, the trains, the buses and many

other types of transportation. Same thing could be said of the police. Energy as a very sophisticated set of apparatus and instrumentation that power cities. You can imagine the amount of data that we might get out of that. And we could say the same thing about health.

And really this is taking place everywhere. In fact, farming is one of the most rapidly transforming industries there is. And when we think about our systems, the systems that power the applications we use today, they look very much like this and they themselves are generating tremendous amounts of data. You can think about the data that's simply the operation of these devices of these components generates in addition to all of the application logic and business applications we put on top of it. Now, we tend to think at times, and you will hear us speak of three tiers, and this is really a practical handle where you have a client, you have a server or something that has some logic, you have some components, and then you have a data store.

Now, you might also hear of the full stack. This is another convenient way to think about applications. You have some front end, you have some back-end logic, something that computes in the background, you have some components, and you have a database, and we call all of this the full stack. However, they still look like the former, much richer, much more diverse, but this is a convenient way to speak about them. Now all of this, both the natural world, both the systems that we built are all generating data and going into data stores. And so, we will be looking at the major building blocks that exist when it comes to data stores.

And you can see here the ones that we will be covering, we have MySQL, Mongo, Redis and Cassandra. Each of these corresponds to a major functional grouping or vertical of types of functionality that you will see in databases. And so, we will be looking at each one of these in much more detail.

Video 2: Relational Databases - Part 1 (04:46)

The relational database management system is the traditional database. That is, it goes back for decades and MySQL is the most widely used relational database management system. So, it's a good one to get familiar with as you will encounter it quite a bit. Now, as you can see here, the way a relational database management lays out the data model, its design of the information in the world that you're trying to capture is that it does so by creating tables and relationships between those tables. More formally, doing a database design involves choosing the tables, the columns and those relationships.

So, if we consider a simple example, say of students and colleges, you can imagine creating a table of students and their information in one of universities and colleges and their information. In database speak, we would talk about entities for the tables and the properties for the columns. Now, there are a few other things that you would include in a relational database design. And as you can see there, one of them would be the identifiers, that is the unique handle for each of those rows and then the relationships between the tables. You can imagine that if you have students and you have colleges, you'd want to have a relationship.

Now, if you want to see a fuller example, go ahead and take a look at the Sakila sample database that is included in many of the MySQL distributions, installations. And if you don't have it, you can search it, download it, and install it for your further study. Now, one of the very first things that we're going to want to do is that we're going to want to connect to that database server. And the way we're going to be doing it is by running a Python program at the command line, as you can see here, and connecting to that database. In order to do that, to write that program, if you don't already have it, go ahead and install Visual Studio Code, then install Python. And once you do that, as you can see here, in addition to the command line prompt that you see there on the laptop and the Python program or runtime that you will need, there's also a driver. That driver captures, within its package, all of the functionality that is needed for you to talk to the database. The one that we will be using, as you can see here in the one that you will need to install is MySQL-connector-Python. Now, once we have installed it, we will confirm that that is available by running a program in Python with just simply one line that simply imports that package.

So, let's go ahead and write that now. So, let's start off by opening up the command line as you can see here. Then I'm going to go ahead and paste that instruction, the same one that we had on the slides and in my installation of Python, because this operating system already has another one and I need to run the most recent one, I will add pip three to that instruction and then I'm going to go ahead and run it. And as you can see there, that package was downloaded and installed.

And now I'm going to go ahead and minimize that window and I'm going to go ahead and drag and drop the folder that I have here on the desktop. Once that opens, you will see that I have a single file here. And once I open that file, I'm going to go ahead and ignore this prompt. You'll see that it has a single line that we saw on the slides and it is simply import in that package. So, I'm

going to go ahead and go back to the command line and I'm going to go ahead and navigate to this folder.

So, I'm going to go ahead and enter here desktop, then sample, I'm going to go ahead and give it permission to access my desktop. And now I'm going to go ahead and list my files to make sure I am in the right directory. And I'm going to go ahead and enter Python 3. And then I'm going to go ahead and enter the driver name. Now, if this works, that means it finds the package, then we're fine. Let's go ahead and run it and as you can see there that executed without an issue. Now, if you see an error, that means there is a problem with your program accessing your driver and you will have to double and debug that. If not, then let's go ahead and go forward. And if you had some problem, make sure you resolve it before you go on.

Video 3: Relational Databases - Part 2 (08:32)

When working with the data store, a database, you'll often hear the term CRUD, that is create, read, update, and delete. And in our case, we will start off here by reading the databases on the server. We will do that in four steps. The first one will be to connect to that database, we will then create a cursor and we will use that cursor to be able to read the databases. And then we will finish up by closing the connections to the database. So, let's go ahead and move to the code. I'm going to go ahead and open up here the show databases file and we'll start off here by adding the driver. We'll then create a connection string.

And this is something that is quite common when connecting to databases or data stores, and a lot of the parameters might be similar. You can see here that I am specifying the user, the password, the location which would be the host. And in this case, I'm leaving the database empty because this is a brand new server and I don't have yet any databases. The only databases will be the ones that ship with the server and the type of authentication. I'm then going to create a cursor and we can use that there, as you can see, and that's part of that driver that we imported on line 1. Then we're going to go ahead and read the databases and this will show us simply the databases that shipped with the server. Because again, this is a new server, we have not added any databases. We're going to go ahead and execute that query and then we're going to fetch all of the results there on line 17 through 18. And then we're going to write them to the console. And we'll end up here by simply cleaning up, that is closing our cursor and closing our connection to the database.

So, I'm going to go ahead and save this, open up the terminal and I'm going to go ahead and show you here that I am in the same directory that file is in and we're going to go ahead and run the file like so. And as you can see there, I have a big error. Now upon reading there my error message, it looks like I have the wrong password which is very likely. I'm going to go ahead and change that. So, I've gone ahead and changed the password. I'm going to go ahead and run it again. Yes. And as you can see there, I have gotten, I have read the default databases. I'm going to go ahead and clear here to simply rerun and as you can see there, we get back the default databases that are on the server. Now before we go forward, note that I have changed my password back to my new pass and I will show you here that we can continue to run the program and we can read the databases once again.

I have also added three files and as you can see here now I have a file called 'create'. We will use this to create a database and a table, then one called 'insert' which we will use to populate that database and that table, and then one called 'selects' which we will simply read what we have input. So, let's go ahead and close and minimize. We will start off by creating the database and table. Just as before, we start by importing the driver, then creating the connection string, then from that creating a cursor. And here we in lines 12 through 14, we're going to drop that database if it exists. And this is something useful to do when you're going to be recreating that database frequently, especially within the development phase. Now once we've done that, we can go ahead and create the database if it doesn't exist, again, just defensive measure here to avoid errors. We will then use that database and this is the instruction that is used to use Pluto, and the name of the database that we're creating is Pluto. Then we're going to go ahead and define a table as you can see there within the query variable and we use that to execute it with the cursor.

Now once we've done that, we've gone ahead, and created the database and within it created the table called POST that only has 2 fields, an ID, and then a stamp as you can see. And then we're going to go ahead and commit those changes in line 34, close the cursor, close the connection. So, let's go ahead and save that. Open up our console here, and we're going to go ahead and run the program. And the program is called create. And as you can see there, that went through without any error. Let's go ahead and take a look at our databases by running the first program we wrote which was show databases. And as you can see there, we now have a database called Pluto.

We're now going to go ahead and write a program to insert into the database. And as before, we start by doing our imports, our drivers first. Then we have datetime which we will use to create a timestamp, then a unique ID. After that we do our connection string and our connection to the database followed by the cursor and here is our code to be able to do the insert into that table. As you can see there, we have now in our connection string here specified Pluto as you can see. Then we create a unique identifier followed by a timestamp, and you can see there the formatting of that timestamp. Then we create the query which is insert into post those two values that we just defined. And then we execute that query. Now the last thing we do is we commit those changes to the database. We close our cursor and our connection. So, let's go ahead and run that now. So, I'm going to go ahead and enter Python 3 and then insert. You can see there, there was no error. I'm going to go ahead and enter a few, as you can see there. Now, that we have data within our database in our table, we're going to go ahead and read that back and we're going to write it into this file called selects.py.

And we'll start off with our driver, then our connection to the database, our cursor, and then we're going to be selecting all of the entries from the table posts. We're going to execute that query, then write to the console anything that we find. And then we're going to go ahead and close our cursor and our connection. I'm going to go ahead and run that program. Now, I'm going to go ahead and clear here and run Python selects. And as you can see there, we can see now all of those entries that we made into that table. We have the unique identifiers followed by the timestamp, and I'm going to go ahead and enter here one more.

Actually, let's do more, two of them, and then I'm going to go ahead and rerun the program. As you can see there, I have two more entries, and if you look closely at the timestamps, you will see there that they are more recent as well. So, as you saw from the programs that we wrote, there's a general pattern that these programs have. We connect to the database, we create a cursor, we write some SQL, the structured query language of relational database management systems, and then we do some clean up. Now, we have shown you a few examples and we will leave the last two for you. We did create, we did read. Now, we will leave it up to you to write an update and a delete. If you are curious about the the nomenclature, the syntax, the SQL that you need to write for MySQL, you can take a look at the documentation and there's plenty of documentation that is available for you. So, try at least one of those. Either update or delete with the tables that we have defined.

Video 4: More About Containers (06:49)

A development machine is a place of a lot of experimentation, and over time that machine can become unstable. You've changed too many configurations, you've installed too many packages, and ultimately you just need to reformat that drive and rebuild that machine, and that's a lot of work. You need to install your operating system, which can take a while to transfer. Put in all the patches, add all of your dependencies for runtimes, languages, and so on. You need to add your data stores, add your application servers and so this can take quite a bit. It can be frustrating; it can really sour your day. Now, when it comes to virtual machines, they really helped the situation quite a bit.

They gave us a foundation upon which we could add additional machines on top of our machine, and this was great for testing. In many ways, they got you closer to what you wanted, although you ultimately needed to add a few more things with most of the stacks that you were working on. However, there is a more fundamental question, and the question is can I run your code? And this turns out to be a difficult question to answer. You're ultimately replicating somebody else's environment. And so as it turns out, one of the frequent responses is your code will not work on my machine.

And one of the things that we've discovered is that shipping code to a server and getting it to run can be hard. And the reason for that is that our software has become more complex. We used to think of the LAMP stack, that is Linux, Apache, MySQL, and PHP or Perl. However, even just looking at the web stack, the picture has become much more complex. And although it might seem surprising, this is a simplification and so it's this next picture. The picture keeps on growing when it comes to hardware, that's true as well. We used to have one type of back end.

Now we have a multitude of devices that we're targeting as well as generalized platforms for computation. So, over time we have more complex software, we have more complex hardware, a varying number of these. And so a group that started to look at this problem created a picture similar to the one that I'm showing you here, where they identified a varying number of software packages even within those, a number of versions and a number of hardware platforms to target as well. And so, what you have is this really challenging scenario where you are hard pressed to be able to replicate somebody else's environment consistently across all of this variability.

And so, as they thought about this problem and considered the history, they thought of shipping.

And you can see here that at one time, being able to ship something meant a varying number of sizes and weights and restrictions, as well as a varying number of ways in which you could ship it. This wasn't an easy problem. The solution that came to this world was the shipping container. And overnight, the world standardized on this box and was able to ship things around the world. And today if you go to any of the port cities, you will see these great snacks of the goods that are being moved around the world.

And so, a similar idea was conceived for software, one where you could package neatly your code in the environment that was provided. That was an extension of the work that had been done in Linux containers but was improved to a degree that it could be standardized and used as an industry abstraction to be able to replicate somebody's environment and run that code. And if you're curious about more of the history and the tremendous work that was done, you can look at this talk that reviews or goes over the history of this development. But ultimately the fundamental difference was that as opposed to loading a whole new machine on top of your machine, as was the case with virtual machines, containers leveraged the fact that you already had an operating system on your machine and simply added the additional executables and libraries, as you can see here, on top of a Docker runtime. So, this had great efficiencies. Not only that, there was a nice clean abstraction to be able to piecemeal, build new, new stacks of technology or new stacks of software. And you can see here, the idea above that base image of how you could add in the case of node if you wanted to, a node and then on top of that be able to use something else like Mongo.

And so, what was done with the success of this abstraction is that the industry adopted it in large numbers and a registry of images was built where today you can navigate to and see some of the packages that are popular being downloaded in the tens of millions and having a very productive ecosystem. More than that, you can define the application that you have built in a very compact text file. As you can see here, in 12 lines of code, I have defined a software application, which is a known application. In these 12 lines of code, I have provided everything that's needed for somebody else to recreate my environment, the operating system, the runtime, which in this case will be node, the code that is required, and once I publish this to the registry, they can pull it with a command and be able to recreate my application. More than that, with extensions such as Docker Compose, you can stitch together an application involving several images like you can see here a database in addition to an application server. So, this was a great success. It has become one of the building blocks in the commercial clouds and companies like Google internally move

billions of containers per week. And it has now become a building block, as I mentioned, to the commercial clouds, independent of which one you're using.

Video 5: Housing Databases in Containers (06:31)

Given that we want to work with multiple databases, the process that we went through to install and configure set up MySQL is quite a lengthy one, and you can imagine that doing that in an organizational context would be even more so. So, what we would like to do is to take this idea of containers and containerize our databases so that they can run on Docker. That is, we want to be able to run this database on top of a platform and be able to access it through our machine as if that were something we went through and configured manually. That is very much the idea of containers.

Now, you could do this yourself. You could configure the application or the database in the application to be able to do that, so that later on if you wanted to reconstruct what you did, it would be easy to do so. And your teammates would certainly appreciate you doing that as well. Now, luckily for us, this idea of being able to create a registry of containers of images that we could leverage is something that has already been done. And if we navigate to this registry Docker Hub, we would be able here to search for anything we'd like. And as you can see there, I have typed in already MySQL and you can see there that that is the very first head.

You can see also that this is an official image, that it has over 10 million downloads, and that if I selected this, I would be able to see more of the documentation, the way in which I could run this, the different versions that are available and so on. Now, we could do the same thing for the other databases that we'd want to explore Mongo, you can see there's several versions. You can see Redis and you can see Cassandra as well. And so, you can see there that instead of building this container images ourselves, we can simply pull them from this registry.

The idea being that we run that, we load it onto our container, and then we access it from our local environment. Now, at this point, if you have not installed Docker, if the machine that you're in doesn't have it, go ahead and install it. Once you do so, go ahead and open up from your menu the dashboard, and once you do, you should see something like the following. As you can see here for me, I have no containers running. This is a clean environment. Now, in order to be able to load that image from the registry onto a container and then run on top of a platform, we need to write a command.

The general pattern of that command is the following. Note there the two braces that I have on top of the command as this being the minimal that you can do. That is doctor run, and then the name of your image. And in our case, we're going to add more arguments. As you can see here, we're going to specify a port, and we're going to be recreating what we did before for MySQL, except this time we're going to be doing it using a container. We're going to go through all of the configuration that we did by hand, but we're going to do it here once again on Docker.

So, you can see there that we are writing Docker run, then the port, the container name, an environment variable that we want to pass in that sets the password. We're going to run this in a detached way. And then the image that we're going to be creating or we're going to be using is that of MySQL. So, as you can see here, I have my container dashboard and I'm going to go ahead and open up the terminal. And then I'm going to go ahead and paste that command that I had on my slides. And as you can see here, we have it once again, the same one Docker run, the port, the name, the variables that we're passing in and the image that we're using.

So, let's go ahead and run this. And as you can see there, that was very fast. And the reason for that is because I already had cached some of the images on my machine and this happens by default. You don't have to do anything to be able to get that. In fact, after the first time that you run it, your machine caches them for you. So, This is why it was faster. I'm going to go ahead now, and delete the images and you will see how much slower that is. And at the same time, I'm simply going to delete my container here, and this is one of the advantages.

If you think about everything we did before, in this case, we can simply get rid of everything we did by simply touching delete here or clicking on 'Delete'. And you can see there that is now gone. Now, I have just deleted the images that I had cached, and now I'm going to go ahead and run that command again. And as you can see there, it couldn't find it. It now is pulling out all of the different pieces that are necessary for that container. And as you can see there, that would take longer. Now, depending on the speed of your connection, this would be very fast as mine, I happen to be in a fast connection or it might take some more time.

But ultimately you can see here that the result is that we get a container. Note that there are a few options here. You can open it up in the browser and, that makes sense or not, depending on the type of container that you're running, you could step into the container, into the command line interface, you could stop the container, restart it, or simply delete it. If we open this up, you can see here some of what was executed, you can take a look at the logs, you could inspect, you

can see here more details about the environment, the mounts, the ports, and as well take a look at the stats. So, as you can see, this approach gives us tremendous speed. We can readily, in one line be able to recreate our entire development environment for the database, we can pull that from the registry, put it into a container and access it through our machine.

Video 6: Interfacing with a Database Container (04:31)

Previously, we installed the database server of MySQL in our local environment, and we did so in the traditional way. That is, you navigate to a site, you download those files, you go through the installation, the configuration and so on, and then we access that server using a Python program. Now, later we did the exact same thing, but we did it using a container that is on top of our Docker platform, we pulled an image from the container registry and we created a new instance of MySQL, but now running in a container. So, let's go ahead and access that container that we have and see if in fact it is the same as accessing a server that was set up in the traditional way, downloading the files and going through the installation. As you can see, I have here my editor and I'm going to go ahead and drag and drop a folder that I have on my desktop and it has a single file inside of a folder called Sample.

I'm going to go ahead and click on it. And as you can see there, it's a pretty simple file, it's code we've used before. The very first block sets the driver that we need to use, then a couple of packages that we need for the code, one is for our timestamps and the other one is a unique identifier. The very first part of it establishes the connection to the database. And if you see, there are no differences to what we did before, it is exactly the same, except now we are running against that database that is inside of a container.

The rest of it is the same, this is the cursor that we create and then the SQL command that we're going to execute. And in this case, we're simply going to list the databases. Then we're going to, if we get some results, we're going to iterate through them and write the results to the console and we'll end up by closing that cursor and that connection. So, let's go ahead and open up the console here, the terminal. As you can see, this happens to be in my desktop, and inside of it I have that single file.

And now I'm going to go ahead and enter python3 show databases. And if I execute that command, you can see there that I get the default databases that are part of that installation. Now, as you can see here, I have opened the docker dashboard. You can see there my container,

MySQL which I have named some MySQL and below it I am in the same location that I was inside of the editor that is on my desktop inside a folder called Sample and inside of it I have 'show databases.' Now, if I enter the same command line instruction that I did before, you can see here that once again I get the databases that we saw before.

Now, however, if I go here and I stop the container and in a moment the color should change there from green to gray and I run this again, you can see there that the container is no longer available. And if I restart it, let me go ahead, and this is already up here. It took a moment and I clicked it twice. Then you can see there that the databases are available once more. So, you can see here that we're running inside of a container environment. You can see here as well that if we simply delete this, then it's gone and you will no longer have access to it.

Everything that you had inside of that container has now been erased. Now, this is quite convenient when it comes to being able to create and iterate quickly through different scenarios and configurations and certainly to be able to create databases at a high speed. So, go ahead and try this yourself. Make sure that you can access that container, make sure that you can stop the container and be able to remove it and recreate it once again.

Video 7: MongoDB: A Document Database (09:24)

At a very high level, a data store is a data store. That is, we store information within it. However, when it comes to the conceptual model that you have to think of as a user of that database, as a developer that is going to be storing information within the database, a relational database management system is very different than a document database. Let's explore a little bit more what we mean by that, especially when it comes to getting started within a database design, a document database is quite different. It is simply a collection of documents, and those documents can have anything you like within them.

Now, that document collection would be the equivalent of a table in a relational database. However, it doesn't have the structure or the schema. And once we get to the code, that will become more apparent. Then you would have within this document, your fields and your properties and this would be the equivalent of columns on a relational database. And ultimately that third part, which is the relationships, is how the collections and the properties interact. Now, there are some great advantages when it comes to document data stores, and that is that the documents are really independent units.

Again, as mentioned previously, you don't have that structure, that hard schema that really limits a lot of what you're doing, especially at the beginning of a project when you are exploring. Then when it comes to the application logic, because of it, as you get going, you don't have to conform to that structure. Now, it might be that you need it and that the functionality of a relational database is really something that you need from the very get go, but this is really within the design space. It depends on what you're trying to do. If you are simply exploring and you're looking to work with a data store, that's really not going to get in your way while you figure things out, document database is perfect.

And as you scale up, it might be that, that continues to be the case. However, there are some trade-offs. So you can see here that one of the biggest advantages is not having that structure pushed upon you. Now, when it comes to thinking about a document, what do we really mean by a document? When you look at this diagram here, you can see that what we really mean is really just a Jason document. You can see here that is a collection of value pairs and what you have within it is really up to you, meaning that you have the flexibility of having different documents that have different properties and different fields within them, even if they're all within the same collection.

Now when it comes to being able to create these, as you might imagine, being able to do so means getting a handle on the database on the table, which in this case is the collection, and then inserting into that. The same thing would be true when it comes to reading back. You get a handle on the database, the collection, and then you pull out what you need. Now, in this case, when it comes to the document database, the one that we're going to be using is MongoDB. This is a widely used, very popular open source database, and once again, we're going to be putting it into a container and accessing it locally on our machine.

The command that we're going to be used to create that Docker container inside of on top of the platform docker is the following. You can see there that we have Docker run at the very end. It's the image name, which is Mongo, and between it we have the port, the name that we're going to give it, and that we're going to run this container, detached. So as you can see here, I have a terminal window opened up at my desktop, and I'm going to go ahead and run that command and I'm going to go ahead and open up the dashboard to be able to see when that is created.

I'm going to go ahead and make this a little bit smaller so you can see, and I'm going to clear my desktop to get rid of that output that you saw there. And now I'm going to go ahead and run the

command. And as you can see there, I am doing a fresh install that was not found locally. I am pulling that container and in a moment, as that gets instantiated and pulled into my local environment, you can see here at the top that I now have a Mongo database up and running and I can go ahead and connect to it.

Now the next thing that we're going to do is that we're going to write some Python code against this new container, this new Mongo database. But before we do so, we need to install the driver. And so I'm going to do precisely that. I'm going to go ahead and clear here the desktop and I'm going to go ahead and install that driver. As you can see there that had already been installed. So I simply get the confirmation that I have already done so. So now let's go ahead and write some code.

I'm going to go ahead and open up my editor. I'm going to go ahead and drag and drop a sample folder that I have on my desktop. And once that opens, you can see there that I have two files. One is called read and the other one is called write. We'll start off with write, and once we do write into the database, then we're going to go ahead and use the second file to be able to read that back. Once we open the file, we're going to go ahead and start off by adding or importing the packages we need. You can see that the very first one there is the driver.

The second one is what we're going to use to create a timestamp that we're going to write into the database as well as one for the unique identifier. Right after that, we're going to go ahead and make a connection and note here that all we're doing is simply writing the name of the database. Then we specify that is in the localhost and the port that you see there is the default port. Right after that, we're going to get a handle on the collection. This is or not the collection, but on the database. And in this case we're going to call it Pluto.

It's worth noting that if that does not exist, it's going to be created. This is a fresh, clean installation of the database, and this is one of those advantages that I mentioned. It creates simply what it does not have. So as opposed to what you see in relational where you need to create things first, create schemas, create tables, and so on, here it'll simply create it for you. Now the next thing after that is going to be the collection of posts under Pluto under that database. And once again, it'll do the same thing. This is the equivalent of the table and it'll simply create the table.

Now next comes where we're going to define a JSON document, and as you can see there, I am creating an ID with the unique ID UUID. And then we're going to go ahead and create a

datestamp. And I am creating a JSON structure there, which is really a dictionary within Python. And then the last thing that we're going to do is that we're going to insert that into the table again, the collection of documents called posts. So let's go ahead and save this document.

Let's go ahead and open up the console. And as you can see there, I am on the desktop. I'm going to go ahead and enter here the name of the file, which is right. And as you can see there, that executed without any errors. So now let's go ahead and write that read file. We'll start off once again by getting a handle on the driver. We'll then make our connection to Mongo. We'll get a handle on the database, then the collections, then we're going to read through all of them. And that will be all that we will be doing.

So let's go ahead and run that program. This one is called Read, and as you can see there, I'm going to go ahead and expand the window. We can see that date stamp that we wrote with that unique identifier. Now, note that Mongo itself adds on its own a unique identifier of its own. And that is what that other item that you see there. Now let's go ahead and make a few more 'Write'. I'm going to go ahead and run two more, and then let's go ahead and read it again. And as you can see there now we have 3 entries.

We have those 3 timestamps, note the time differences there. We have 3 unique IDs that we created, plus the ones that Mongo itself is creating. So now it's your turn. Make sure you can create the document database container. Make sure you can connect to it, make sure you can write, make sure you can read.

Video 8: Redis: A Key-Value Database (07:08)

When it comes to database functionality, one of the most featureful, one of the ones that has the most things in it, even the name is very long, is relational database management systems. Now, at the other end of the spectrum, the simplest possible, the one that has the less friction for you to use would be key-value databases. You can almost think of them as the type of variable declaration that you do in your program, but in this case, you're getting the benefits of a server. When it comes to the database that we're going to be using, we're going to be using the open source project called 'Redis'. And this is an in-memory key-value pair data store, a database. It is very performant as you can imagine, it goes into memory, not all the way down to disk. So, it has those tremendous advantages of speed. And because of it, it is frequently used as an additional layer on top of other databases. So, let's say you're using MySQL. If you wanted to get more

speed, more performance, you would add an additional layer using Redis. You could do the same thing with a document data store as well. And so this is one that is frequently used at other times that is used purely as a database that is with nothing else.

Now, in our case, we're going to do the same thing that we did with the RDBMS, that we did with the document store. In this case, we're going to create a container of the key-value pair data store, and in this case, we're going to do that with Redis. You can see here the command that we're going to enter, we have Docker run. The image is going to be Redis, the port is 6379, and the container name is going to be some Redis, and we're going to run that container detached.

So, as you can see here, I have my Docker dashboard, I've cleared out all my images and then I've gone ahead and pasted the same command instruction there that we had in our slides. Now, I'm going to go ahead and create it. You can see there that it is pulling from the Docker registry and it has created that instance. You can see here also within my dashboard that is up and running. Now that we've created our container, I'm going to go ahead and drag and drop a folder from my desktop into my editor, and it looks like that did not go through. Let me try again.

And it looks like I already have that open. I'm going to go ahead and close it just to make sure I have the right one. You can see there that the color at the bottom of the editor turns purple. Now, I'm going to go ahead and drag and drop. Let's go ahead and open up here the file. So, we can see that we have two files. The very first one is the write file. We'll read there in alphabetical order, but the first one we're going to work on is going to be write, and we will be writing to that Redis container.

And then we're going to use the second one to read what we write in. So, let's go ahead and open up that file. And as you can see here, the very first thing that we get is that we get the driver and we need to install that driver first. Let's go ahead and do that. Now. I'm going to grab there, pip install Redis, and then going to enter that at the command line. And as you can see in this environment, that has already been installed. If that wasn't the case, you would see the installation at that point.

So, let's go ahead and close the console and let's go on to the next line. And as you can see there, I am now using that driver to connect to the database. Note that I'm connecting locally, so that's localhost to my local container. The default port is 6379 and my database is the database zero. Again, very simple, very trim. The next thing that we're going to do is that we are going to create

a timestamp, as you can see there, as we have done before, and now we're going to push that value into the database.

Note that we are not defining schema, we're not defining tables, collections, anything else. We're simply writing into the database, and in this case, under the handle of nums. Let's go ahead and save that file. Open up the console and let's go ahead and run that file. That's right. And let me make sure I have python3 here. And as you can see there, that went through. Now, let's go ahead and take a look at the read file. As you can see there we start once again with the driver. The next thing that we do is the connection to the database.

Then after that, we're simply reading the content that we have within it. So, let's go ahead and save the file. Let's go ahead and open up the console. In this case, I'm going to go ahead and run 3, I mean, Python 3 read. And as you can see there, we've written, we've read back the value that we added. So, let me go ahead and do a few more writes. And now let's go ahead and read those back. And as you can see there, we get the values. So, out of the databases that we have tried, you can see here that this is by far the simplest, the less syntax to write, the less options that are involved with creating and storing some information and retrieving it as well.

So, now it's your turn. Make sure that you can create the containers, make sure you can write and read from them. You might also want to try some of the visual managers for this, the graphical user interfaces for this database. This is the one that is available for Redis, you can download it and install it and run it locally and take a look inside of your database. Now, just as a reminder, this is a very performant database and it is one that is used frequently on top of all other databases to improve scalability, to improve speed, and to have a better user experience.

It is also one of the friendliest when it comes to the developer experience and being able to store information. Now, not everything needs to be a simple key-value pair where the value of that key is a simple string. You can have a more complex structure, so you could map the type of rich information that you store within document databases or within a relational database into the structure. And depending on your needs, this is something that is done sometimes. So, once again, now it is your turn.

Video 9: Cassandra: A Distributed Scalable Database (09:43)

A relational database management system gives you a tremendous amount of functionality, but it also gives you a tremendous amount of structure. Document databases, on the other hand,

give you a lot of flexibility, and as you grow up, you might need some of that structure. On the other hand, when it comes to key value data stores, we have tremendous speed in many of the implementations. Now what about scalability and being able to scale with ease? We can make the comparison here to relational database management systems that do pretty well at low scale. But as you scale, there are some scalability challenges that certainly can be done transparently, that is without you really thinking about it. Now when it comes to that last need, the need to be able to scale with ease, Cassandra was set up with that in mind.

So, it is not going for the features of the relational database management systems, nor the speed or flexibility, but for scalability with ease and it makes sense that it was introduced initially by Facebook and that they learned from the lessons of the first implementations that really went for that large scale, which were databases such as Amazon's Dynamo or Google's Big table, where they were looking at reliable performance data and in other cases, just rich data models that were able to grow without a lot of user intervention.

And so in this case, as you can see here, simply put, the goal is for organizations to use it to handle large data, volumes of data in a distributed fashion, and you can see here some of those points being emphasized that you have a master list model that is decentralized. So, you can tolerate failures even up to a datacenter if you have multiple others around the world. If you have, if you have the need to be able to scale, you can simply address it with hardware and when it comes to the hardware, it doesn't have to be specialized. It's commodity servers and that's a great thing to have, simply adding servers instead of people. People are more expensive than machines now and you have a master list replication. You don't have to worry about being having to maintain and scale that master on it's own. And so you can get a sense of the scalability that we're talking about here is a benchmark made by Netflix and you can see here that they're writing at the rate of over a million writes per second on 288 nodes across several clusters. I've provided the link there at the bottom of the slide in case you want to read more about it. So, with that in mind, as before, we're going to go ahead and create a container of this large scalable database, but we're going to do so local in the same way we have done before for others.

So, as you can see, here is the command to be able to do so. You can see Docker run, the name of the image is Cassandra, the port is 9042, and the container name is Cassandra and we're going to run that detached. So, let's go ahead and paste that command onto our terminal and let's go ahead and run it and as you can see there it is pulling that image, creating a local instance and I'm

going to go ahead and open up the dashboard here so we can see when that image comes, it's instantiated and then is ready to go and as you can see there, we now have the green color of our container, meaning that that container is up and running and it's healthy. So, now let's go ahead and close our dashboard and let's go ahead and exit from our terminal and let's go ahead and open up Visual Studio. Into it I'm hardware and to go ahead and drag and drop a folder that I have with a couple of files, as we have done before. One of them will be to write to Cassandra, the other one will be to read from it. So, let's go ahead and get started with the write. We're going to start off by installing the container.

So, we're going to go ahead and copy and paste onto our command line here and as you can see there in my local environment, I have already installed that. So, I get simply the confirmation. I'm going to go ahead and close the terminal and we're going to continue with our code here. You can see there that I am importing the driver and then making a reference to datetime as we have done previously as well and now I'm going to make a connection here to the database. And note that I'm using a term here called keyspace, you can think of that as the database in the context of Cassandra. I'm then going to create something that looks like SQL and it's pretty close to SQL as you can see there and I am creating a key space if no, if it does not exist, and I'm going to call it Pluto and here this is the equivalent of being able to create a database, if that database does not exist.

We're then going to set for the session the key space of Pluto, and that would be the equivalent on relational traditional SQL of saying use in the name of the database. We're then going to create a table and note that we're executing into the database, into the session on each of those blocks. In this case, I'm simply going to create a table and then once I do that, I'm going to go ahead and write into that table and that table is going to have two columns, as you can see there, and the first one is simply going to be a dummy string that I'm going to be writing in and then I'm going to have the timestamp.

So, let's go ahead and save this and now I'm going to go ahead and open up my console. Once again, let's go ahead and clear. Let's go ahead and run that program, Python 3, and then I'm going to use the write file and as you can see there, that was written into the Cassandra database. So, now let's go ahead and read from that database. I'm going to go ahead and open up the read file. We'll start off here with our driver. We'll then make our connection to the database and set Pluto as the key space, the database that we're going to be using. Then we're

going to read all of the rows of the table post, and then we will write those to the console. So, I'm going to go ahead and save the file, open up my console, and then I'm going to go ahead and run, in this case, the read file and as you can see there, we have read successfully that row back the one that we wrote before. Let's go ahead and write three times to that database and then let's go ahead and read it and as you can see there, we get the timestamps that we put in back as expected. So, now it's your turn. Go ahead and walk through the code. Make sure you can replicate what we have done, that you can create your containers, that you can run the code. In addition to that, we want you to go ahead and explore a different type of client. In the past we have made recommendation for visual managers, that is a visual graphical user interface that you can use to connect to the database and see the components in a visual way. In this case, we will ask you to log into the container and start up a shell and use what is called the cqlsh and in this case that's a cryptic way of saying the SQL interface onto this database.

So, let me go ahead and show you how that is done. Using the container dashboard, you'll see that there is an icon that allows you to do that, and this is a graphical way to do it. You can also just do everything purely through the command line. But as you can see here, there are a few options, and the second one is to open up the CLI and as you can see there, that launches a new window and you're now inside of the container at the command prompt. Now if we go ahead and enter that CQL shell and hit return, you can see there that we're now at the prompt for Cassandra. Now we can go ahead and say use Pluto and as you can see there when I forgot the semicolon to end my command there, you can see there that you now are using Pluto. At this point we could enter, select star from post, and once again, I did not enter the semicolon and you can see there that we would get back the timestamps that we did previously by running a Python program. So, once again, it is now your turn. Go ahead and dig in, try the code, build the containers, log into the containers and run your shell commands here at the command prompt.

Video 10: Serverless Cloud Database Overview (04:00)

By now, you have a pretty good sense of databases. You've used the traditional relational database, you've used document databases, you've used key value store databases and distributed scalable databases. But you might be wondering about how this looks like in the cloud and perhaps of buzzwords such as serverless. What does that mean? Well, let's take a step back and consider where we have been and what we have done. We started out by doing a local installation in the traditional way. Navigate to a website, download a program, run it locally and

configure it. And really this picture of what you see here is a conceptual one, because really what we did is we downloaded that database, installed it, ran it locally, and then built some program to access the database.

We ran a Python program that used the driver to be able to access and interact with the database. Now, recognizing all of the complexity that that entails, we took a shortcut and we used containers, and those containers could be loaded from a registry of images and then run locally on top of our Docker runtime. Now, you can imagine that you could do the same thing in the cloud. Within the cloud, you can provision a machine and on top of that add the Docker runtime, and on top of that add those containers or those images and run containers locally.

Very much the same model that we're using locally, but now in the cloud. And certainly that gives us a lot of advantages. It certainly gives us portability, but it would be nice to have more. What if the cloud provider could do those installations for you and simply offer you the service? That is, you wouldn't have to think about what operating system it was installed on. You wouldn't have to think about things such as scalability. You would simply use the database offering and let the cloud provider solve those problems. Well, that exists and it's called 'Manage Cloud Services', and it's something that might be a great resource for you to use if it fits your use case.

Now, what if we wanted more, if we were greedy and we wanted even more simplicity? So, this is what we refer to when we think about serverless. That is, this scenario in which you no longer are defining the operating system, neither the database that is being used. You're simply being offered functionality of a database through an API application programming interface, so there's no decision on whether you will use a document or a relational or a key value or other type of data store. That is an implementation detail that the cloud vendor decides on.

And all you really care about is that the information you are inputting and recalling meets a certain level of service level agreement. And if that is matched, then this is this is the performance you want. And in some scenarios this is a perfect solution. It lets you bypass everything else that we have considered. Now, in this last example, when it comes to databases, we will be looking precisely at that scenario and we will be using a cloud database, a serverless database called Firebase. And you can navigate here to that address firebase.google.com. Within it, once you access that address, you can go to the console and add a new project. Now, once we are inside of the project, we will be creating a database and then just like we did before, from our

local environment, we will be accessing that database that is running in the cloud in a serverless model.

Video 11: Firebase: A Serverless Cloud Database (04:59)

Firebase, as the name indicates, it's a back-end service or a bass. And practically what we're going to be doing here is we're going to be using that back end as a database, it's storage, and we're going to be connecting to it from Python. So, as you can see here, I have created a database project, and I am at the console. I'm going to go ahead and click on 'Project Overview' and 'Project Settings.' You can see here some of the information for this project. Now, we're going to need some permissions to connect from Python. And we can get that through service accounts. And if you select here the 'Python' option, you can generate a new private key that will be a file that you can download and save.

And as you can see here on my listing of files, I have done precisely that. I won't share it because I don't want to delete the project afterwards, but all you have to do is simply add it to your project and then as you will see, reference it from the code. So, once you have done that, once you have created your credentials and you have created your project within Firebase, now the next thing to do is to go ahead and add the package that you need from Python. And so, in this case, we're going to use pip3 installer, and we're going to go ahead and install Firebase admin package.

So, as you can see there, and yes, I left out the install. So, I'm going to go ahead and enter 'Install'. And as you can see there, I have made the installation. Now once we do that, then we can import credentials and enter also the database functionality. Now at this point here on line seven, you can see here is where you would reference the path to the credentials that you downloaded from Firebase, the site. And then, you'd need to initialize the app with your service account and, point it to the database that you're working with.

And so here, if we go to the Cloud Firestore, we can see here some of the services that exist. And in this case, we're going to be using the real time database as you can see here. And this is the path that we need. That would also, if you take a look at your credential's files, you'd be able to see that information there as well. And so, once we have that, we can go ahead and continue here on our file. And you can see here that I am referencing, this is a hierarchical database, think of JSON.

And you can see there that I am referencing the root of the information that I'm going to construct. And then I'm going to create a child called users. And to that, we're going to go ahead and add the data that you see here, which is a couple of keys, and you can see there are some of the properties. Now below that I'm simply going to go ahead and read some of that information back. And once I have it, well, actually, no, let me see that. That's going to be an update. And then on the last one, I'm going to go ahead and read that data back and then just write it to the console.

Now, as you can see here, when I was testing, I created all of that information. And what I'm going to do is that I'm going to go ahead and delete it. And this is now gone, as you can see. Now, let's go ahead and go back to the code. And let's go ahead and run. It's going to be python3. Anyway, my file is Fire. And so, as you can see there, I have written into the database, and I am now reading it back. And if we take a look at the real time database through the browser, you can see here that information has in fact been recreated. So, there you have it, a pretty fast and quick way to store information, in this case JSON information onto the database. You can see there that once we installed the package, it was just a matter of setting up the credentials, identifying the database that we're going to write into, then adding some structure for the data that you're pushing into.

You can see here how to update and this block and then after that, we're simply going to read it back and writing it to the console. So, this structure in this database is a pretty friendly way to get started with cloud storage without much structure in being able to pull and retrieve some information.

Video 12: Benefits of Serverless Architecture (03:49)

By now, you have done multiple types of installations and you've worked locally, you've worked with the cloud and you've worked with containers and you've worked with variations of these types of configurations and architectures. If we look at the top left, this is your traditional environment where you're doing the installation locally, where you're doing the traditional 3-tier application. If you look at the next slide over, you can see there that we're doing a lot of the same, but we're doing it now in the cloud, we have additional tiers. Now, you could take a variation of that and run some of those components in the cloud, but now in virtual machines, as you can see there.

Now, we took an even more advanced example and we ran that on top of containers using a container registry. Now, we discussed about how you could take some of the products that are offered in the cloud that have been packaged in a very convenient way in which you transfer some of the responsibility to the cloud provider and you use a managed cloud service, and a managed cloud database is a great idea and something that is very convenient to leverage in many conditions. Now, it might be that you're even greedier and you want even more simplicity, in which case you might want to try a serverless database as the one we just tried.

Now, it might interest you to know that when it comes to the impact on the organization and the amount of people that are involved, the decision making, the coordination that needs to be had when it comes to realizing and using these technologies in a project. As you look there from left to right, you can see that the teams are much smaller, they're much more focused, and they require less coordination since you have less people now, it makes sense that on the left hand side, when organizations were still building their own data centers and they had large teams and they had to solve a lot of the problems that ultimately have been addressed in a much more sophisticated way by the cloud providers, and they have done so all over the world, you have to replicate a lot of that locally. Now, as time went on and we got some better building blocks like virtual machines, the teams became smaller and we used those as templates to be able to move faster. By the time containers were mature enough for us to be able to use, we had a great abstraction there to build on top of.

And as you saw within our course, you were able to span a great deal of databases with minimal effort when it came to the configuration and being able to do an implementation. As you can see there, we can then deploy in seconds; we can live for minutes as we did many times or leave it around for hours. But we have that flexibility, we can do it in demand, we can do it on demand, we can do it programmatically. Now, when it comes to that last tier server list, you can see there that you do not need a large team. There's a lot of configurations that have been hidden from you. There's a lot of convenience that is available there as well. Now, if this is a good fit for what you're trying to achieve, and then you have a pretty small team that could take care of this type of scenario in architecture.

So, with that in mind, I hope you have enjoyed the database section. There's a great deal of richness here. There's a great deal of products that are used the world over by some of the largest organization, and there's a lot here to dig more into. And so, we will leave that up to you.

Video 13: Types of Databases: Review (02:51)

We have seen five different types of databases. We saw the relational model, the document model, the key-value pair model, and in fact Cassandra is a key value pair data store. However, it is known more for it's scalability, it's decentralization, it's ability to grow without really having a master, and because of it, we emphasize those aspects. Now, finally we saw one of the new up and coming models when it comes to the Cloud, one of these serverless kinds of databases with Firebase. Now, as you might imagine, there are a great deal more. You can see here that large collection, a large collection of those, and there are many more than that.

And if you're wondering what kind of categories these would fall into, here are some broad categories in addition to the ones we saw. So, we certainly saw relational, which would be SQL. We also saw the NoSQL which are all the other ones, and you can see there are a few more analytical time series, embedded, big data, full-texts, and graph databases. Now, if you're wondering about where to look into for the innovation the cutting edge, the Apache Software Foundation is a good place to go and explore. You can see there that I've given you the URL. It is apache.org.

And just so you get a sense here of what they do, it is not only data type projects, there are also many others as you can see there in the categories. There are over 202 projects, in fact and if we were to look just at a couple of the ones that are data related, we could take a look at databases. There are 25 projects that you can go look at, and if you looked at the big data ones, you can see there that there are 50 such type projects. So, a lot for you to go there and explore. Obviously, you can navigate to places like GitHub which has another good set of database and data-related type projects.

But this is a good one to go look at because a lot of the big, big packages that are used heavily in the market as well as some of the workflow engines and others you can see there. Most of those in those big lists are the Apache Software Foundation. So, as you can see, the data universe of products, it's alive and thriving. There are a great deal of needs in the market, and the innovation is just starting to address them. This area is going to grow significantly in the years to come. So it's a wonderful thing to be connected to and to track, and it's also a lot of fun to keep seeing that continuous innovation.